

CENTRO UNIVERSITÁRIO MAURÍCIO DE NASSAU
CURSO DE CIÊNCIA DA COMPUTAÇÃO

DOCUMENTAÇÃO BACK-END (NODE.JS)

Desenvolvimento do Back-end para o projeto do site Not Fat

Responsável

João Carlos Oliveira da Silva - 01806272

Disciplina: Códigos de Alta Performance - Web
Professor(a): Anderlan Henrique Batista Siqueira

Maceió - AL
2026

Resumo

Este código é a API do Back-End do sistema, desenvolvida em Node.js com o Express. A função dele é atuar como o servidor que liga o aplicativo (Front-End) ao banco de dados MySQL de um jeito seguro.

Basicamente, o código gerencia três partes principais do projeto:

Autenticação: O login é feito integrando com a conta do Google. O código valida o acesso da pessoa e, se for o primeiro acesso dela no app, já cria o cadastro no banco de dados automaticamente.

Diário Alimentar: É a parte que controla as refeições. A API possui rotas para buscar a lista de alimentos disponíveis, salvar o que o usuário comeu (junto com a quantidade exata) e apagar itens que ele tenha colocado por engano no prato.

Água e Perfil: Controla o usuário em si. Permite que ele atualize seu nome no aplicativo e gerencia o consumo de água diário, salvando cada registro e calculando o histórico da semana.

Para fazer tudo isso funcionar direito, o arquivo também configura a segurança básica. Ele usa variáveis de ambiente para esconder as senhas do banco de dados, libera as permissões corretas para o navegador não bloquear o app e gerencia a conexão com o banco para que o sistema responda mais rápido.

1 Funções do Back-End

1.1 Configurações Iniciais e Dependências

`require()`: Importa ferramentas e arquivos para o projeto. Funciona como a base para adicionar funcionalidades externas ao código.

`config()`: Carrega variáveis secretas (como senhas do banco de dados) de um arquivo seguro, garantindo que informações sensíveis não fiquem visíveis no código.

`express()`: Cria o servidor principal da aplicação, responsável por receber as requisições do sistema e organizar o caminho dos dados.

`OAuth2Client()`: Faz a ponte com a API do Google, validando de forma segura os logins dos usuários através de tokens.

`app.use()`: Ativa os "filtros" do servidor, aplicando regras padronizadas para todas as requisições antes que elas cheguem às rotas finais.

`cors()`: Regra de segurança que permite a comunicação entre o sistema visual (Front-End) e o servidor (Back-End), evitando que o navegador bloqueie os acessos.

`express.json()`: Traduz os dados enviados pelo usuário para um formato estruturado, permitindo que o sistema consiga ler e manipular as informações facilmente.

`mysql.createPool()`: Mantém várias conexões abertas com o banco de dados. Isso agiliza a resposta do sistema e evita travamentos quando várias pessoas acessam o app ao mesmo tempo.

1.2 Rotas e Regras de Negócio

`respondServerError`: Função de proteção. Ela oculta erros internos complexos e devolve uma mensagem simples ao usuário, evitando o vazamento de detalhes técnicos do servidor.

`POST /login`: Rota de entrada e cadastro. Valida o acesso pelo Google e verifica o banco de dados: se for um usuário novo, cria a conta automaticamente; se já existir, apenas libera a entrada.

`POST /adicionar`: Rota de diário alimentar. Grava qual alimento o usuário escolheu, a refeição exata e a quantidade consumida.

`GET /alimentos`: Rota de catálogo. Busca a lista completa de comidas cadastradas no sistema para preencher os menus de seleção do aplicativo.

`GET /usuario/:idUsuario`: Rota de consulta de perfil. Puxa os dados básicos de um usuário (como nome e e-mail) utilizando apenas o código de identificação dele.

GET /refeicao/:idUsuario/:idRefeicao: Rota de consolidação de prato. Lista tudo o que foi consumido em uma refeição e, de forma inteligente, já soma as quantidades de alimentos registrados repetidamente.

DELETE /refeicao/.../:idAlimento: Rota de correção. Permite apagar do banco de dados um alimento específico que foi adicionado por engano a uma refeição.

PUT /usuario/:idUsuario: Rota de edição de perfil. Atualiza as informações cadastrais do usuário, permitindo a troca do nome registrado no sistema.

POST /agua: Rota de hidratação. Registra o consumo de água do usuário no momento da ação, anotando a quantidade exata em mililitros.

GET /agua/:idUsuario: Rota de histórico e gráficos. Calcula o volume total de água consumido no dia atual e traz o levantamento dos dias anteriores para acompanhamento.

1.3 Inicialização do Servidor

process.env.PORT || 3000: Definição da porta de rede. O sistema tenta identificar automaticamente a porta configurada pelo ambiente de hospedagem (através das variáveis de ambiente). Caso não exista uma configuração prévia, ele adota a porta 3000 como o padrão seguro para rodar localmente.

app.listen(): Execução do servidor. É o comando que efetivamente liga o sistema e o coloca "no ar". Ele instrui a aplicação a ficar aguardando requisições na porta definida, permitindo receber conexões de qualquer interface de rede (regra definida pelo IP 0.0.0.0). Por fim, exibe uma mensagem de confirmação no terminal indicando que a API foi iniciada com sucesso.